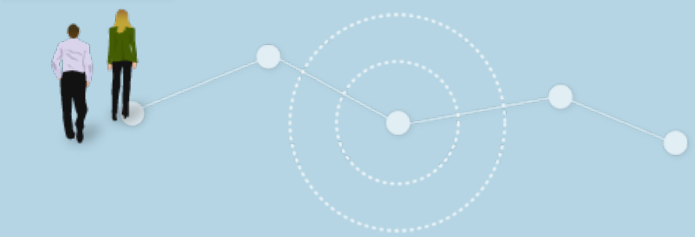


관계 데이터 모델

MySQL로 배우는 데이터베이스 개론과 실습

관계 데이터 모델의 개념

1. 릴레이션
2. 릴레이션 스키마와 인스턴스
3. 릴레이션의 특징
4. 관계 데이터 모델



1. 릴레이션

- 릴레이션(relation) : 행과 열로 구성된 테이블, 엑셀의 표

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000

- Relation은 사전적인 의미로 관계, 친척 등을 의미하므로 이 모델을 사용하는 데이터 베이스는 관계 데이터베이스

- 대부분의 DBMS는 Relation Database Management System(RDBMS)

1. 릴레이션

도서 1, 축구의 역사, 굿스포츠, 7000
도서 2, 축구아는 사람, 나무수, 13000
도서 3, 축구의 이해, 대한미디어, 22000
도서 4, 골프 바이블, 대한미디어, 35000
도서 5, 피겨 교본, 굿스포츠, 8000



도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000

그림 2-1 **데이터와 테이블(릴레이션)**

도서번호 = **{1, 2, 3, 4, 5}**
도서이름 = **{축구의 역사, 축구아는 사람, 축구의 이해, 골프 바이블, 피겨 교본}**
출판사 = **{굿스포츠, 나무수, 대한미디어}**
가격 = **{7000, 13000, 22000, 35000, 8000}**

→ 첫 번째 행(1, 축구의 역사, 굿스포츠, 7000)의 경우 네 개의 집합에서 각각 원소 한 개씩 선택하여 만들어진 것으로 이 원소들이 관계(relationship)를 맺고 있다.

1. 릴레이션

❖ 관계(relationship)

❶ 릴레이션 내에서 생성되는 관계 : 릴레이션 내 데이터들의 관계

❷ 릴레이션 간에 생성되는 관계 : 릴레이션 간의 관계

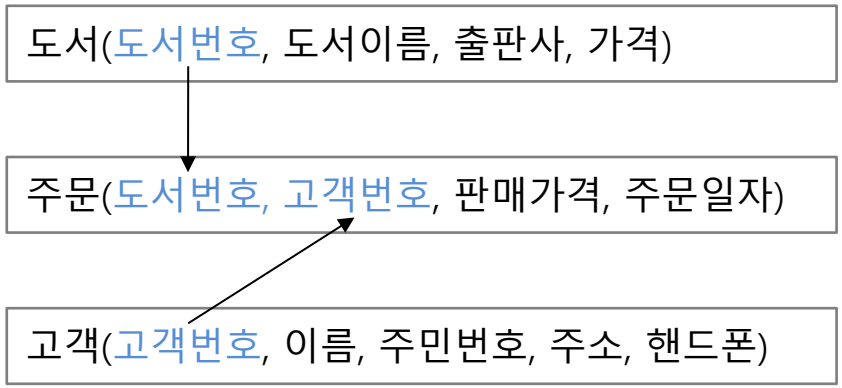
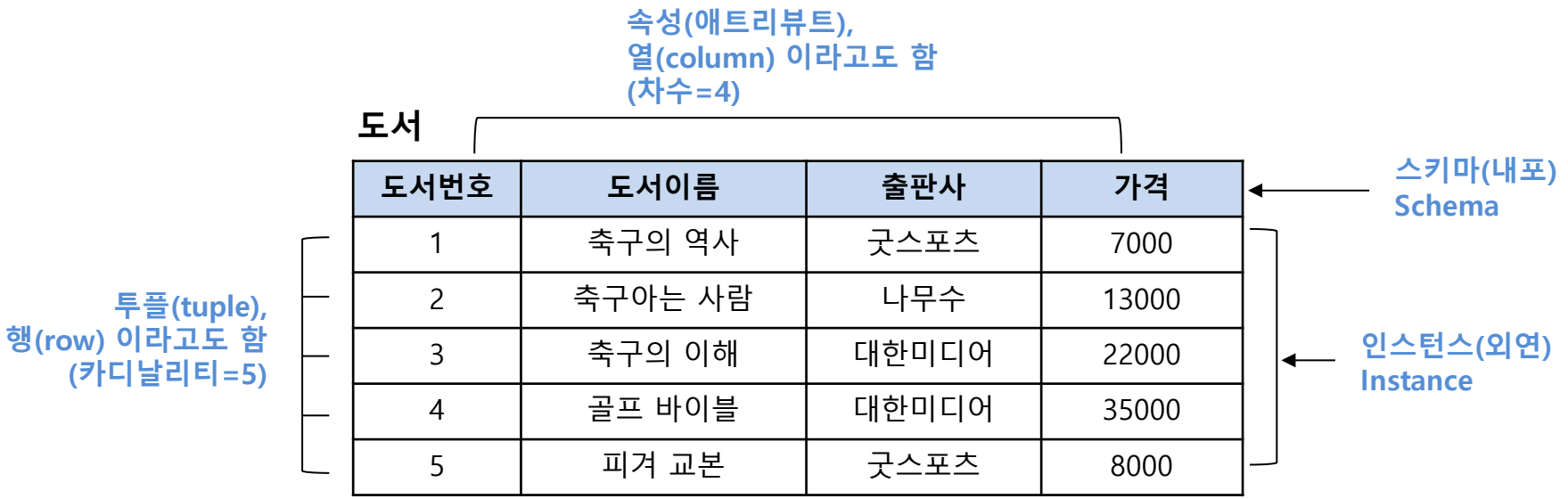


그림 2-2 **릴레이션 간의 관계**

2. 릴레이션 스키마와 인스턴스



도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000

그림 2-3 **도서 릴레이션**

2. 릴레이션 스키마와 인스턴스

❖ 릴레이션 스키마

■ 스키마의 요소

- 속성(attribute) : 릴레이션 스키마의 열
- 도메인(domain) : 속성이 가질 수 있는 값의 집합
- 차수(degree) : 속성의 개수

■ 스키마의 표현

- 속성 이름만 표기 하거나, 속성과 도메인을 같이 표기
- 릴레이션 이름(속성1, 속성2, 속성3, ...)
- 릴레이션 이름(속성1 : 도메인1, 속성2 : 도메인2, 속성3 : 도메인3, ...)

  EX) 도서(도서번호, 도서이름, 출판사, 가격)

2. 릴레이션 스키마와 인스턴스

❖ 릴레이션 인스턴스

■ 인스턴스 요소

- 투플(tuple) : 릴레이션의 행
- 카디날리티(cardinality) : 투플의 수

→ 투플이 가지는 속성의 개수는 릴레이션 스키마의 차수와 동일하고,
릴레이션 내의 모든 투플들은 서로 중복되지 않아야 함

표 2-2 릴레이션 구조와 관련된 용어

릴레이션 용어	같은 의미로 통용되는 용어	파일 시스템 용어
릴레이션(relation)	테이블(table)	파일(file)
스키마(schema)	내포(intension)	헤더(header)
인스턴스(instance)	외연(extension)	데이터(data)
투플(tuple)	행(row)	레코드(record)
속성(attribute)	열(column)	필드(field)

3. 릴레이션의 특징

❶ **속성은 단일 값을 가진다**

각 속성의 값은 도메인에 정의된 값만을 가지며 그 값은 모두 단일 값이어야 함

❷ **속성은 서로 다른 이름을 가진다**

속성은 한 릴레이션에서 서로 다른 이름을 가져야만 함

❸ **한 속성의 값은 모두 같은 도메인 값을 가진다**

한 속성에 속한 열은 모두 그 속성에서 정의한 도메인 값만 가질 수 있음

❹ **속성의 순서는 상관없다**

속성의 순서가 달라도 릴레이션 스키마는 같음

예) 릴레이션 스키마에서 (이름, 주소) 순으로 속성을 표시하거나 (주소, 이름) 순으로 표시하여도 상관없음

3. 릴레이션의 특징

❺ 릴레이션 내의 중복된 투플은 허용하지 않는다

하나의 릴레이션 인스턴스 내에서는 서로 중복된 값을 가질 수 없음,

즉 모든 투플은 서로 값이 달라야 함(집합과 같다)

(축구의 역사, 굿스포츠, 7000)

(축구의 역사, 굿스포츠, 7000)

❻ 투플의 순서는 상관없다

투플의 순서가 달라도 같은 릴레이션임. 관계 데이터 모델의 투플은 실제적인 값을 가지고 있으며 이 값은 시간이 지남에 따라 데이터의 삭제, 수정, 삽입에 따라 순서가 바뀔 수 있음

3. 릴레이션의 특징

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구 아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
5	피겨 교본	굿스포츠	8000
6	피겨 교본, 피겨 기초	굿스포츠	8000

그림 2-4 **릴레이션의 특징에 위배된 경우**

3. 릴레이션의 특징

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구 아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
5	피겨 교본	굿스포츠	8000
6	피겨 교본, 피겨 기초	굿스포츠	8000

동일한 투플이 중복되면 안 됨

속성의 값은 단일 값이어야 함

그림 2-4 릴레이션의 특징에 위배된 경우

4. 관계 데이터 모델

- 관계 데이터 모델은 데이터를 2차원 테이블 형태인 릴레이션으로 표현함
- 릴레이션에 대한 제약조건과 관계 연산을 위한 관계대수를 정의함

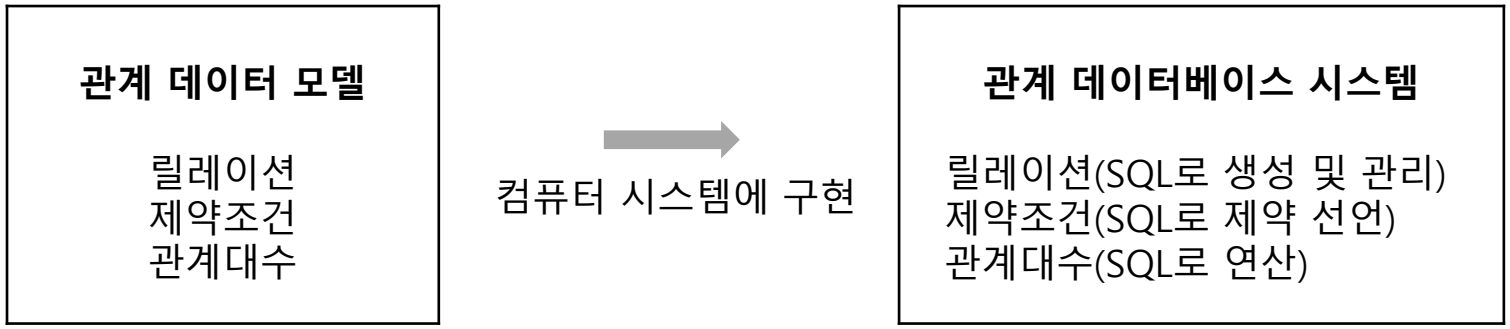
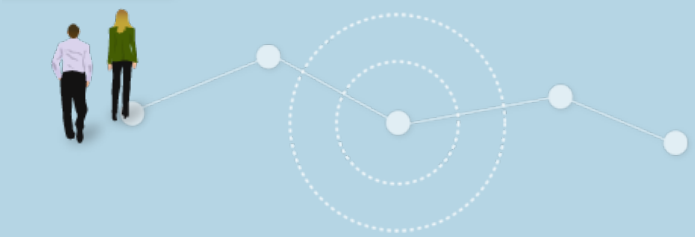


그림 2-5 **관계 데이터베이스 시스템**

무결성 제약조건

1. 키
2. 무결성 제약조건
3. 무결성 제약조건의 수행



1. 키

- 특정 투플을 식별할 때 사용하는 속성 혹은 속성의 집합
- 릴레이션은 중복된 투플을 허용하지 않음 → 각각의 투플에 포함된 속성들 중 어느 하나(혹은 하나 이상)는 값이 달라야 함. 즉 키가 되는 속성(혹은 속성의 집합)은 반드시 값이 달라서 투플들을 서로 구별할 수 있어야 함
- 키는 릴레이션 간의 관계를 맺는 데도 사용됨



그림 2-6 자동차 한 대당 키는 단 하나

1. 키

고객

고객번호	이름	주민번호	주소	핸드폰
1	박지성	810101-1111111	영국 맨체스타	000-5000-0001
2	김연아	900101-2222222	대한민국 서울	000-6000-0001
3	장미란	830101-2333333	대한민국 강원도	000-7000-0001
4	추신수	820101-1444444	미국 클리블랜드	000-8000-0001

도서

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000

주문

고객번호	도서번호	판매가격	주문일자
1	1	7000	2014-07-01
1	2	13000	2014-07-03
2	5	8000	2014-07-03
3	2	13000	2014-07-04
4	4	35000	2014-07-05
1	3	22000	2014-07-07
4	3	22000	2014-07-07

그림 2-7 마당서점 데이터베이스

1. 키

❖ 슈퍼키

■ 투플을 유일하게 식별할 수 있는 하나의 속성 혹은 속성의 집합

투플을 유일하게 식별할 수 있는 값이면 모두 슈퍼키가 될 수 있음

(고객 릴레이션 예)

- 고객번호 : 고객별로 유일한 값이 부여되어 있기 때문에 투플을 식별할 수 있음
- 이름 : 동명이인이 있을 경우 투플을 유일하게 식별할 수 없음
- 주민번호 : 개인별로 유일한 값이 부여되어 있기 때문에 투플을 식별할 수 있음
- 주소 : 가족끼리는 같은 정보를 사용하므로 투플을 식별할 수 없음
- 핸드폰 : 한 사람이 여러 개의 핸드폰을 사용할 수 있고 반대로 핸드폰을 사용하지 않는 사람이 있을 수 있기 때문에 투플을 식별할 수 없음

■ 고객 릴레이션은 고객번호와 주민번호를 포함한 모든 속성의 집합이 슈퍼키가 됨

EX) (주민번호), (주민번호, 이름), (주민번호, 이름, 주소), (주민번호, 이름, 핸드폰),
(고객번호), (고객번호, 이름, 주소), (고객번호, 이름, 주민번호, 주소, 핸드폰) 등

1. 키

❖ 후보키

- **투플을 유일하게 식별할 수 있는 속성의 최소 집합**

(고객 릴레이션 예)

- 고객번호 : 고객번호는 고객 테이블에서 고객 한명 당 유일한 값
- 주민번호 : 주민번호는 나라에서 부여한 유일한 값이므로 고객 테이블내에서도 유일함(Unique)

(주문 릴레이션 예)

- 고객번호 : 한 명의 고객이 여러 권의 도서를 구입할 수 있으므로 후보키가 될 수 없고,
  고객번호가 1인 박지성 고객은 세 번의 주문 기록이 있으므로 투플을 유일하게 식별할 수 없음
- 도서번호 : 도서번호가 2인 '축구아는 사람'는 두 번의 주문 기록이 있으므로 투플을 유일하게 식별할 수 없음

- **주문 릴레이션의 후보키는 2개의 속성을 합한 (고객번호, 도서번호)가 됨**
- **2개 이상의 속성으로 이루어진 키를 복합키(composite key)라고 함**

1. 키

❖ 기본키

- **여러 후보키 중 하나를 선정하여 대표로 삼는 키**
- **후보키가 하나뿐이라면 그 후보키를 기본키로 사용하면 되고, 여러 개라면 릴레이션의 특성을 반영하여 하나를 선택하면 됨**

- **기본키 선정 시 고려사항**
  - 릴레이션 내 투플을 식별할 수 있는 고유한 값을 가져야 함.
  - NULL 값은 허용하지 않음.
  - 키 값의 변동이 일어나지 않아야 함.
  - 최대한 적은 수의 속성을 가진 것이라야 함.
  - 향후 키를 사용하는 데 있어서 문제 발생 소지가 없어야 함.

- **릴레이션 스키마를 표현할 때 기본키는 밑줄을 그어 표시함**

  릴레이션 이름(속성1, 속성2, ... 속성N)

  EX) 고객(고객번호, 이름, 주민번호, 주소, 핸드폰)

  도서(도서번호, 도서이름, 출판사, 가격)

1. 키

❖ 대리키

- 기본키가 보안을 요하거나, 여러 개의 속성으로 구성되어 복잡하거나, 마땅한 기본키가 없을 때는 일련번호 같은 가상의 속성을 만들어 기본키로 삼는 경우가 있음 이러한 키를 대리키(surrogate key) 혹은 인조키(artificial key)라고 함
- 대리키는 DBMS나 관련 소프트웨어에서 임의로 생성하는 값으로 사용자가 직관적으로 그 값의 의미를 알 수 없음

주문

주문번호	고객번호	도서번호	판매가격	주문일자
1	1	1	7000	2014-07-01
2	1	2	13000	2014-07-03
3	2	5	8000	2014-07-03
4	3	2	13000	2014-07-04
5	4	4	35000	2014-07-05
6	1	3	22000	2014-07-07
7	4	3	22000	2014-07-07

그림 2-8 대리키를 사용하도록 변경된 주문 릴레이션

1. 키

❖ 대체키

- 기본키로 선정되지 않은 후보키
- 고객 릴레이션의 경우 고객번호와 주민번호 중 고객번호를 기본키로 정하면 주민번호가 대체키가 됨

1. 키

❖ 외래키

- **다른 릴레이션의 기본키를 참조하는 속성을 말함**
- **다른 릴레이션의 기본키를 참조하여 관계 데이터 모델의 특징인 릴레이션 간의 관계(relationship)를 표현함**

- **외래키의 특징**
  - 관계 데이터 모델의 릴레이션 간의 관계를 표현함
  - 다른 릴레이션의 기본키를 참조하는 속성임
  - 참조하고(외래키) 참조되는(기본키) 양쪽 릴레이션의 도메인은 서로 같아야 함
  - 참조되는(기본키) 값이 변경되면 참조하는(외래키) 값도 변경됨
  - NULL 값과 중복 값 등이 허용됨
  - 자기 자신의 기본키를 참조하는 외래키도 가능함
  - 외래키가 기본키의 일부가 될 수 있음

1. 키

고객

고객번호	이름	주민번호	주소	핸드폰
1	박지성	810101-1111111	영국 맨체스타	000-5000-0001
2	김연아	900101-2222222	대한민국 서울	000-6000-0001
3	장미란	830101-2333333	대한민국 강원도	000-7000-0001
4	추신수	820101-1444444	미국 클리블랜드	000-8000-0001

기본키

도서

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000

기본키

참조

외래키

참조

주문

주문번호	고객번호	도서번호	판매가격	주문일자
1	1	1	7000	2014-07-01
2	1	2	13000	2014-07-03
3	2	5	8000	2014-07-03
4	3	2	13000	2014-07-04
5	4	4	35000	2014-07-05
6	1	3	22000	2014-07-07
7	4	3	22000	2014-07-07

기본키

그림 2-9 릴레이션 간의 참조 관계

1. 키

❖ 외래키

- 외래키 사용 시 참조하는 릴레이션과 참조되는 릴레이션이 꼭 다른 릴레이션일 필요는 없음. 즉 자기 자신의 기본키를 참조할 수도 있음

참조

기본키 외래키

선수번호	이름	주소	멘토번호
1	박지성	영국 맨체스타	NULL
2	김연아	대한민국 서울	3
3	장미란	대한민국 강원도	4
4	추신수	미국 클리블랜드	NULL

그림 2-10 멘토 릴레이션

1. 키

❖ 요약

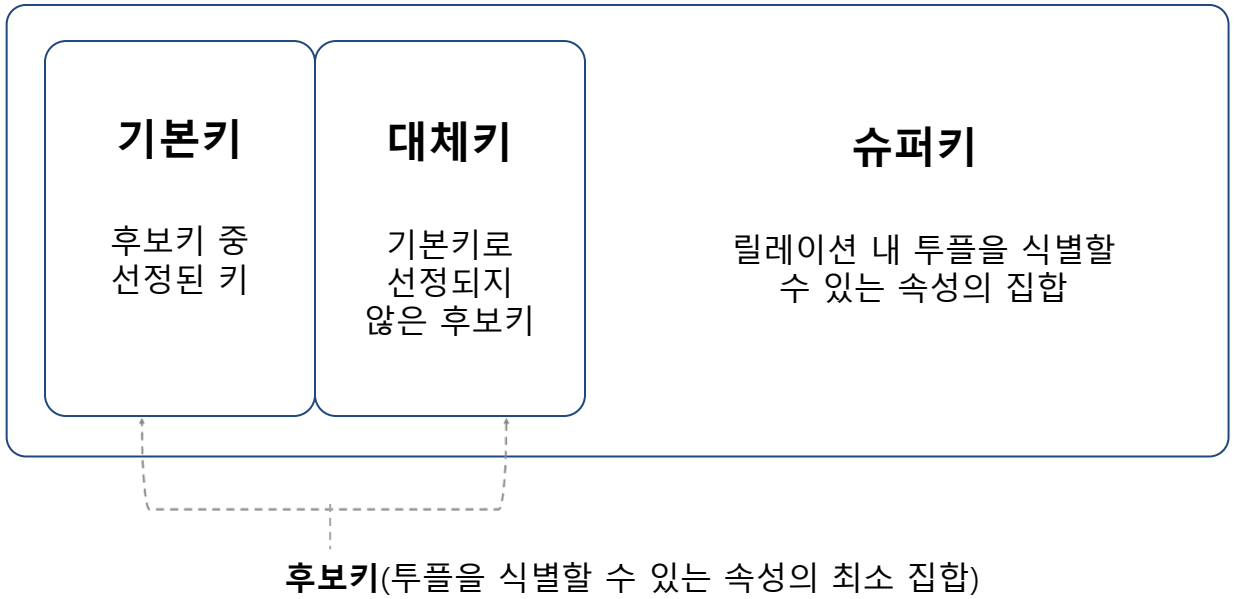


그림 2-11 **키의 포함 관계**

2. 무결성 제약조건

- **데이터 무결성(integrity, 無缺性)**

  데이터베이스에 저장된 데이터의 일관성과 정확성을 지키는 것, 여러가지 제약조건으로 지켜짐

- **도메인 무결성 제약조건**

  도메인 제약(domain constraint)이라고도 하며, 릴레이션 내의 투플들이 각 속성의 도메인에 지정된 값만을 가져야 한다는 조건. SQL 문에서 데이터 형식(type), 널(null/not null), 기본 값(default), 체크(check) 등을 사용하여 지정할 수 있음.

- **개체 무결성 제약조건**

  기본키 제약(primary key constraint)이라고도 함. 릴레이션은 기본키를 지정하고 그에 따른 무결성 원칙, 즉 기본키는 NULL 값을 가져서는 안 되며 릴레이션 내에 오직 하나의 값만 존재해야 한다는 조건임.

- **참조 무결성 제약조건**

  외래키 제약(foreign key constraint)이라고도 하며, 릴레이션 간의 참조 관계를 선언하는 제약조건임. 자식 릴레이션의 외래키는 부모 릴레이션의 기본키와 도메인이 동일해야 하며, 자식 릴레이션의 값이 변경될 때 부모 릴레이션의 제약을 받는다는 것임

2. 무결성 제약조건

- **무결성 제약 조건의 수행 시점**
  - 데이터 변경의 종류
  - CRUD, ABCD

이름 (CRUD)	이름 (ABCD)	조작	SQL
Create	Add	생성	INSERT
Read, Retrieve	Browse	읽기	SELECT
Update	Change	갱신	UPDATE
Delete, Destroy	Delete	삭제	DELETE

2. 무결성 제약조건

표 2-3 제약조건의 정리

구분	도메인	키	
	도메인 무결성 제약조건	개체 무결성 제약조건	참조 무결성 제약조건
제약 대상	속성	투플	속성과 투플
같은 용어	도메인 제약 (Domain Constraint)	기본키 제약 (Primary Key Constraint)	외래키 제약 (Foreign Key Constraint)
해당되는 키	-	기본키	외래키
NULL 값 허용 여부	허용	불가	허용
릴레이션 내 제약조건의 개수	속성의 개수와 동일	1개	0~여러 개
기타	• 투플 삽입, 수정 시 제약사항 우선 확인	• 투플 삽입/수정 시 제약사항 우선 확인	• 투플 삽입/수정 시 제약사항 우선 확인 • 부모 릴레이션의 투플 수정/삭제 시 제약사항 우선 확인

2. 무결성 제약조건

❖ 도메인 무결성 제약조건 위반

- 시험점수가 최대값 100점을 넘어서 150점 인 경우
- 주민번호가 14자리인 경우. 230316-3698019 -> 6 + 7 = 13
- 주문일자가 2023-03-34 or 2023-13-01

2. 무결성 제약조건

❖ 개체 무결성 제약조건

- 삽입 : 기본키 값이 같으면 삽입이 금지됨
- 수정 : 기본키 값이 같거나 NULL로도 수정이 금지됨
- 삭제 : 특별한 확인이 필요하지 않으며 즉시 수행함

학번	이름	학과코드
501	박지성	1001
401	김연아	2001
402	장미란	2001
502	추신수	1001

그림 2-12 학생 릴레이션

(501, 남슬찬, 1001) ↓ 삽입 거부

학번	이름	학과코드
501	박지성	1001
401	김연아	2001
402	장미란	2001
502	추신수	1001

(NULL, 남슬찬, 1001) ↓ 삽입 거부

학번	이름	학과코드
501	박지성	1001
401	김연아	2001
402	장미란	2001
502	추신수	1001

그림 2-13 개체 무결성 제약조건의 수행 예 (기본키 충돌 및 NULL 값 삽입)

2. 무결성 제약조건

❖ 참조 무결성 제약조건

■ 삽입

- 학과(부모 릴레이션) : 투플 삽입한 후 수행하면 정상적으로 진행된다.
- 학생(자식 릴레이션) : 참조받는 테이블에 외래키 값이 없으므로 삽입이 금지된다.

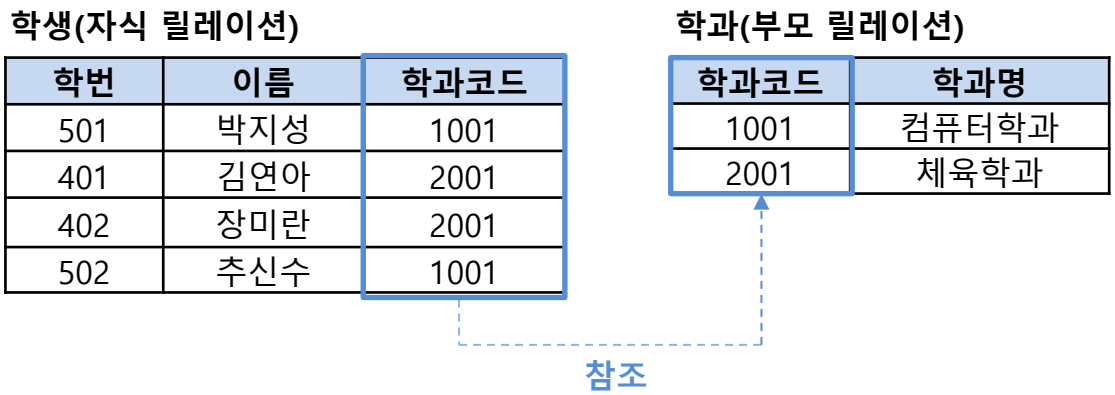


그림 2-14 **학생관리 데이터베이스**

2. 무결성 제약조건

❖ 참조 무결성 제약조건

■ 삭제

- 학과(부모 릴레이션) : 참조하는 테이블을 같이 삭제할 수 있어서 금지하거나
  다른 추가 작업이 필요함
- 학생(자식 릴레이션) : 바로 삭제 가능함

※ 부모 릴레이션에서 투플을 삭제할 경우 참조 무결성 조건을 수행하기 위한 고려사항

❶ 즉시 작업을 중지
❷ 자식 릴레이션의 관련 투플을 삭제
❸ 초기에 설정된 다른 어떤 값으로 변경
❹ NULL 값으로 설정

2. 무결성 제약조건

❖ 참조 무결성 제약조건

■ 수정

- 삭제와 삽입 명령이 연속해서 수행됨.
- 부모 릴레이션의 수정이 일어날 경우 삭제 옵션에 따라 처리된 후 문제가 없으면 다시 삽입 제약조건에 따라 처리됨.

2. 무결성 제약조건

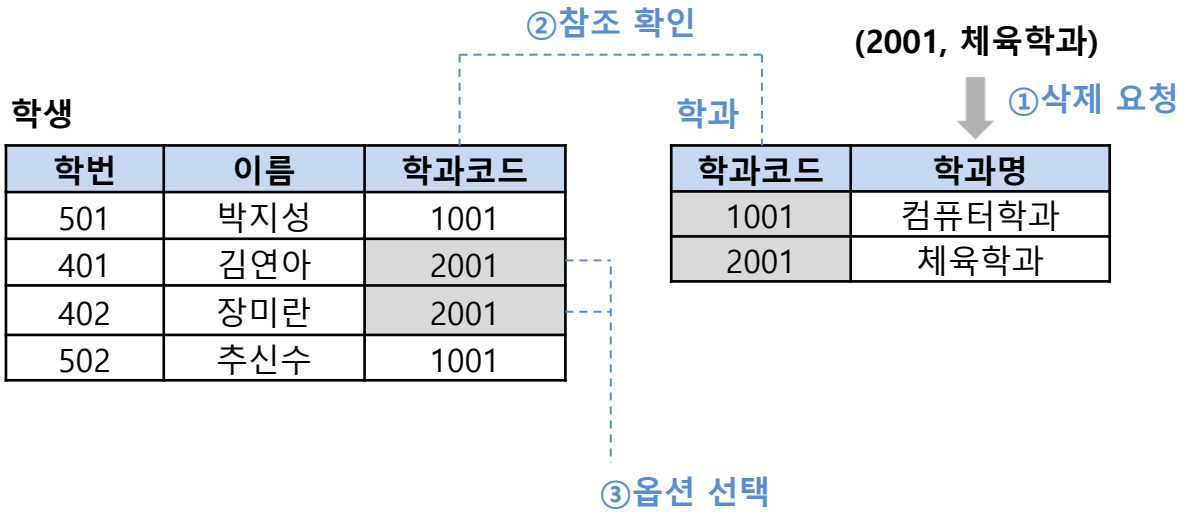
❖ 참조 무결성 제약조건

표 2-4 참조 무결성 제약조건의 옵션(부모 릴레이션에서 투플을 삭제할 경우)

명령어	의미	예
RESTRICTED	자식 릴레이션에서 참조하고 있을 경우 부모 릴레이션의 삭제 작업을 거부함	학과 릴레이션의 투플 삭제 거부
CASCADE	자식 릴레이션의 관련 투플을 같이 삭제 처리함	학생 릴레이션의 관련 투플을 삭제
DEFAULT	자식 릴레이션의 관련 투플을 미리 설정해둔 값으로 변경함	학생 릴레이션의 학과가 다른 학과로 자동 배정
NULL	자식 릴레이션의 관련 투플을 NULL 값으로 설정함(NULL 값을 허가한 경우)	학과 릴레이션의 학과가 NULL 값으로 변경

2. 무결성 제약조건

❖ 참조 무결성 제약조건



① RESTRICTED : 요청한 삭제 작업중지(에러 처리)
② CASCADE : 학생 릴레이션의 해당 투플을 같이 연쇄적으로 삭제(CASCADE)
③ 기본값으로 변경(미리 설정한 값, DEFAULT)
④ NULL 값으로 설정

그림 2-15 **참조 무결성 제약조건에서 부모 릴레이션의 투플을 삭제할 경우**

무결성 제약조건

고객(customer)

고객번호	이름	주민번호	주소	핸드폰
1	박지성	810101-1111111	영국 맨체스타	000-5000-0001
2	김연아	900101-2222222	대한민국 서울	000-6000-0001
3	장미란	830101-2333333	대한민국 강원도	000-7000-0001
4	추신수	820101-1444444	미국 클리블랜드	000-8000-0001

기본키

참조

외래키

주문

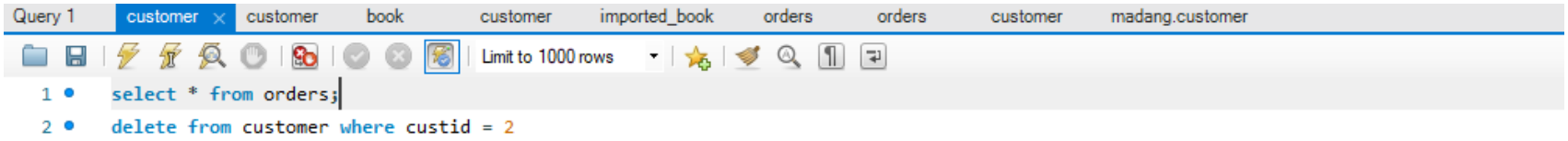
주문번호	고객번호	도서번호	판매가격	주문일자
1	1	1	7000	2014-07-01
2	1	2	13000	2014-07-03
3	2	5	8000	2014-07-03
4	3	2	13000	2014-07-04
5	4	4	35000	2014-07-05
6	1	3	22000	2014-07-07
7	4	3	22000	2014-07-07

기본키

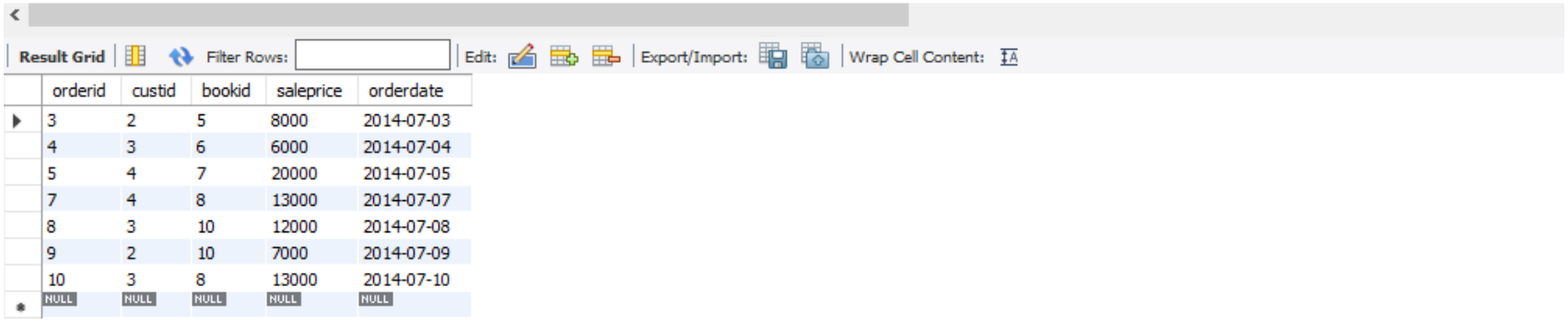
DELETE 시도

그림 2-9 릴레이션 간의 참조 관계

무결성 제약 조건 위반

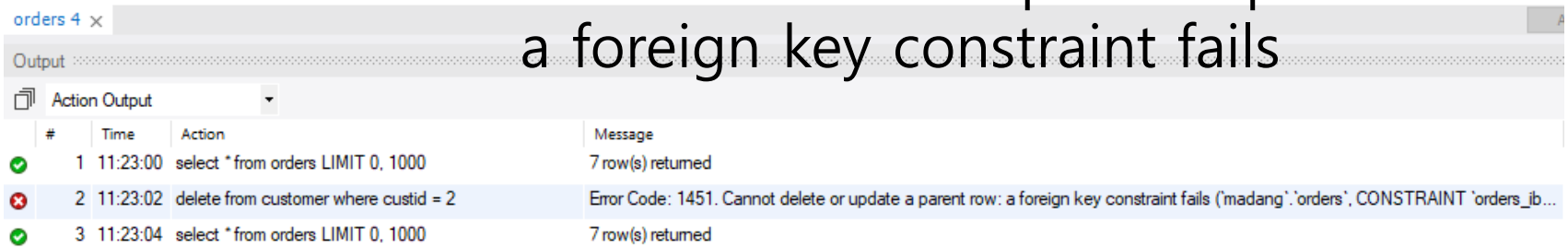


DELETE FROM customer WHERE custid = 2;



orderid	custid	bookid	saleprice	orderdate
3	2	5	8000	2014-07-03
4	3	6	6000	2014-07-04
5	4	7	20000	2014-07-05
7	4	8	13000	2014-07-07
8	3	10	12000	2014-07-08
9	2	10	7000	2014-07-09
10	3	8	13000	2014-07-10
NULL	NULL	NULL	NULL	NULL

cannot delete or update a parent row: a foreign key constraint fails



#	Time	Action	Message
1	11:23:00	select * from orders LIMIT 0, 1000	7 row(s) returned
2	11:23:02	delete from customer where custid = 2	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (`madang`.`orders`, CONSTRAINT `orders_ib...
3	11:23:04	select * from orders LIMIT 0, 1000	7 row(s) returned

MySQL과 MariaDB

- MySQL은 1995년 발표된 오픈소스 DBMS로 스웨덴의 MySQL AB사 제품
- 이후 썬 마이크로시스템사에 인수 그 이후 오라클사에 인수
- 인수 이후 오픈소스 기반이였으나 MySQL에 비호의적
- 주 개발자였던 마이클 몬티 와이드니우스(Michael Monty Widenius)가 회사를 나와서 MySQL 5.5 기반에서 MariaDB라는 오픈소스 버전을 출시

MySQL	
최초 개발사	MySQL AB
개발사	오라클
발표일	1995년 5월 23일
라이선스	GPL v2 (커뮤니티 에디션) 상용 라이선스 (표준, 엔터프라이즈, 클러스터)
링크	

MariaDB	
개발자	MariaDB 재단
발표일	2009년 10월 29일
계열	MySQL
라이선스	GPL v2 LGPL v2.1 (클라이언트 라이브러리)
홈페이지	
소스 코드	



SQL 기초

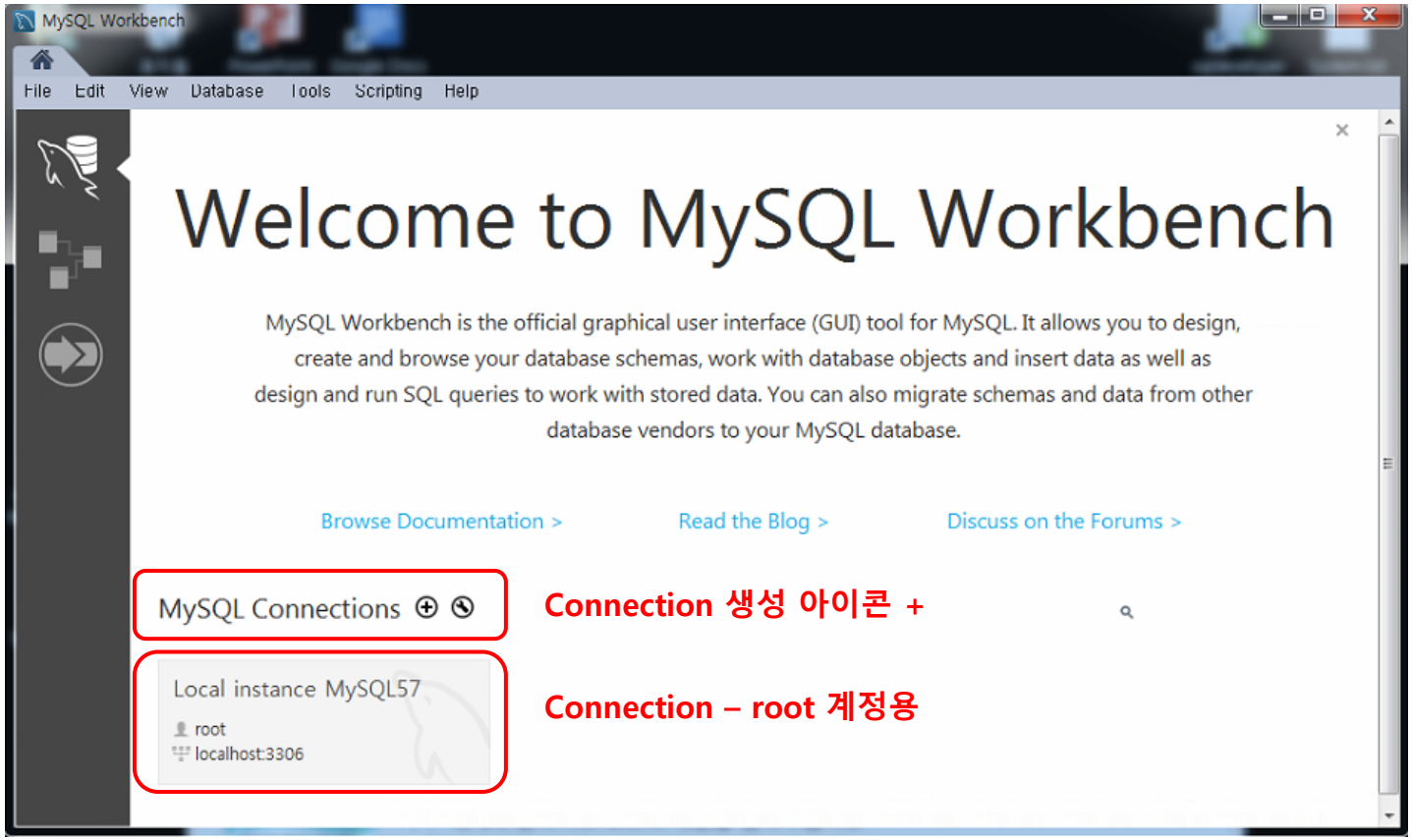
MySQL로 배우는 데이터베이스 개론과 실습

실습 자료

- 실습자료

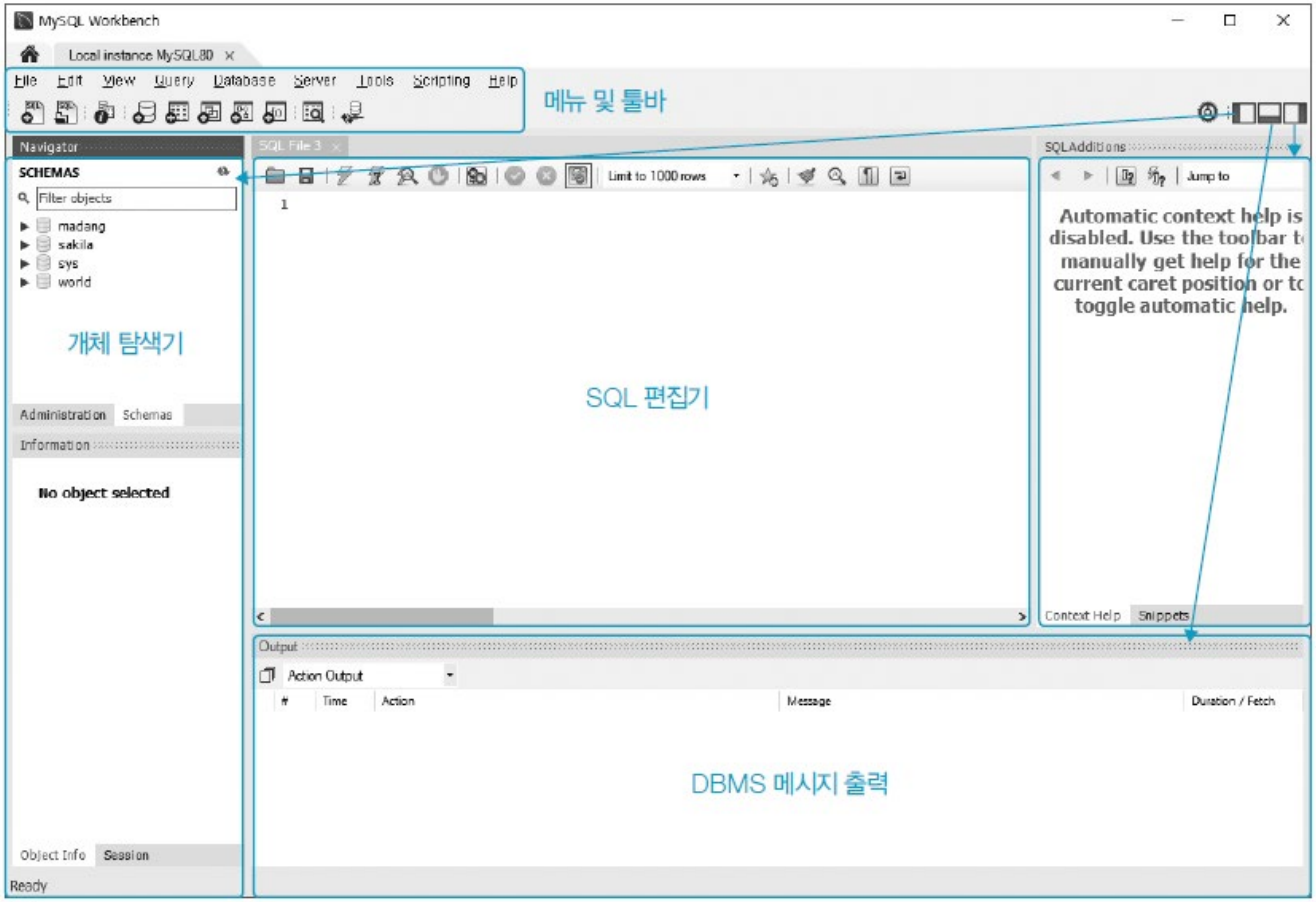
MySQL 연습 데이터 베이스 생성

❖ MySQL Workbench



MySQL 연습 데이터 베이스 생성

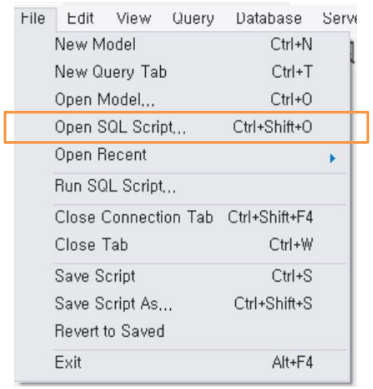
❖ MySQL Workbench 시작



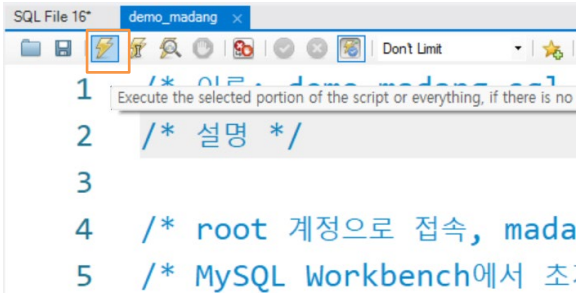
[그림 3-9] Workbench SQL 개발 화면

MySQL 연습 데이터 베이스 생성

- SQL script 파일 다운 로드
- 강의 자료 링크에서 20250318/practice2.sql 다운로드
- workbench 실행



- Open SQL Script 메뉴 선택 > practice2.sql 선택



- script 실행 버튼 클릭



MySQL 연습 데이터 베이스

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

Book 테이블

orderid	custid	bookid	saleprice	orderdate
1	1	1	6000	2014-07-01
2	1	3	21000	2014-07-03
3	2	5	8000	2014-07-03
4	3	6	6000	2014-07-04
5	4	7	20000	2014-07-05
6	1	2	12000	2014-07-07
7	4	8	13000	2014-07-07
8	3	10	12000	2014-07-08
9	2	10	7000	2014-07-09
10	3	8	13000	2014-07-10

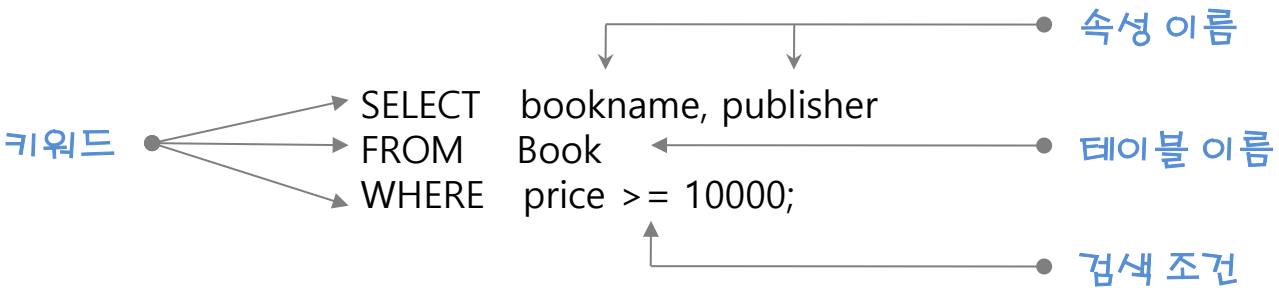
Orders 테이블

custid	name	address	phone
1	박지성	영국 맨체스타	000-5000-0001
2	김연아	대한민국 서울	000-6000-0001
3	장미란	대한민국 강원도	000-7000-0001
4	추신수	미국 클리블랜드	000-8000-0001
5	박세리	대한민국 대전	NULL

Customer 테이블

SELECT 문

❖ SELECT 문의 구성요소



❖ SELECT 문의 기본 문법

```
SELECT [ALL | DISTINCT] 속성이름(들)
FROM          테이블이름(들)
[WHERE        검색조건(들)]
[GROUP BY     속성이름]
[HAVING       검색조건(들)]
[ORDER BY     속성이름 [ASC | DESC]]
```

[ ] : 대괄호 안의 SQL 예약어들은 선택적으로 사용한다.
| : 선택 가능한 문법들 중 한 개를 사용할 수 있다.

SELECT 문

❖ 쿼리 실행 순서

```
SELECT [ALL | DISTINCT] 속성이름(들)
FROM          테이블이름(들)
[WHERE        검색조건(들)]
[GROUP BY     속성이름]
[HAVING       검색조건(들)]
[ORDER BY     속성이름 [ASC | DESC]]
--------------------------------------------------------------------------------
[ ] : 대괄호 안의 SQL 예약어들은 선택적으로 사용한다.
| : 선택 가능한 문법들 중 한 개를 사용할 수 있다.
```

- ❖ FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY
- ❖ ASC: 오름차순, DESC: 내림차순
- ❖ DISTINCT: 중복 제거

SELECT 문

❖ SELECT/FROM_서점에 어떤 도서가 있는지 알고 싶다

질의 3-1 모든 도서의 이름과 가격을 검색하시오.

```
SELECT    bookname, price
FROM      Book;
```

bookname	price
축구의 역사	7000
축구아는 여자	13000
축구의 이해	22000
골프 바이블	35000
피겨 교본	8000
역도 단계별기술	6000
야구의 추억	20000
야구를 부탁해	13000
올림픽 이야기	7500
Olympic Champions	13000

(질의 3-1 변형) 모든 도서의 가격과 이름을 검색하시오.

```
SELECT    price, bookname
FROM      Book;
```

price	bookname
7000	축구의 역사
13000	축구아는 여자
22000	축구의 이해
35000	골프 바이블
8000	피겨 교본
6000	역도 단계별기술
20000	야구의 추억
13000	야구를 부탁해
7500	올림픽 이야기
13000	Olympic Champions

SELECT 문

❖ 주석

- **코드에 목적, 동작 방식등의 설명을 추가해 가독성을 향상**
  - 한 줄 주석(#)

```
1   # 이것은 한 줄 주석입니다.
2 • select * from book;
```

  - 한 줄 주석(--)
    - -- 다음에 공백(띄어쓰기)가 필요

```
1   -- 이것은 한 줄 주석입니다.
2 • select * from book;
```

  - 여러 줄 주석(/* ... */)

```
1  /*
2      이것은 여러줄 주석입니다.
3      책 목록 전체를 쿼리합니다.
4  */
5 • select * from book;
```

SELECT 문

❖ SELECT/FROM_서점에 어떤 도서가 있는지 알고 싶다

질의 3-2 모든 도서의 도서번호, 도서이름, 출판사, 가격을 검색하시오.

```
SELECT    bookid, bookname, publisher, price
FROM      Book;
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

```
SELECT    *
FROM      Book;
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

SELECT 문

❖ SELECT/FROM_서점에 어떤 도서가 있는지 알고 싶다

질의 3-3 도서 테이블에 있는 모든 출판사를 검색하시오.

```
SELECT    publisher
FROM      Book;
```

publisher
굿스포츠
나무수
대한미디어
대한미디어
굿스포츠
굿스포츠
이상미디어
이상미디어
삼성당
Pearson

※ 중복을 제거하고 싶으면 DISTINCT라는 키워드를 사용한다.

```
SELECT    DISTINCT publisher
FROM      Book;
```

publisher
굿스포츠
나무수
대한미디어
이상미디어
삼성당
Pearson

❖ WHERE 조건_가격이 20,000원 미만인 도서가 무엇인지 알고 싶다

표 3-5 WHERE 절에 조건으로 사용할 수 있는 술어

술어	연산자	예
비교	=, <>, <, <=, >, >=	price < 20000
범위	BETWEEN	price BETWEEN 10000 AND 20000
집합	IN, NOT IN	price IN (10000, 20000, 30000)
패턴	LIKE	bookname LIKE '축구의 역사'
NULL	IS NULL, IS NOT NULL	price IS NULL
복합조건	AND, OR, NOT	(price < 20000) AND (bookname LIKE '축구의 역사')

SELECT 문

❖ WHERE 조건_가격이 20,000원 미만인 도서가 무엇인지 알고 싶다

■ 비교

질의 3-4 가격이 20,000원 미만인 도서를 검색하시오.

```
SELECT    *
FROM      Book
WHERE     price < 20000;
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

SELECT 문

❖ WHERE 조건_가격이 20,000원 미만인 도서가 무엇인지 알고 싶다

■ 범위

질의 3-5 가격이 10,000원 이상 20,000 이하인 도서를 검색하시오.

```
SELECT    *
FROM      Book
WHERE     price BETWEEN 10000 AND 20000;
```

bookid	bookname	publisher	price
2	축구아는 여자	나무수	13000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
10	Olympic Champions	Pearson	13000

※ BETWEEN은 논리 연산자인 AND를 사용할 수 있다.

```
SELECT    *
FROM      Book
WHERE     price >= 10000 AND price <= 20000;
```

bookid	bookname	publisher	price
2	축구아는 여자	나무수	13000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
10	Olympic Champions	Pearson	13000

SELECT 문

❖ WHERE 조건

■ 집합

출판사가 '굿스포츠' 혹은 '대한미디어'인 도서를 검색하시오.

```
SELECT    *
FROM      Book
WHERE     publisher IN ('굿스포츠', '대한미디어');
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000

※ 출판사가 '굿스포츠' 혹은 '대한미디어'가 아닌 도서를 검색하시오.

```
SELECT    *
FROM      Book
WHERE     publisher NOT IN ('굿스포츠', '대한미디어');
```

bookid	bookname	publisher	price
2	축구아는 사람	나무수	13000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000
NULL	NULL	NULL	NULL

SELECT 문

❖ WHERE 조건

■ 패턴

'축구의 역사'를 출간한 출판사를 검색하시오.

```
SELECT      bookname, publisher
FROM        Book
WHERE       bookname LIKE '축구의 역사';
```

bookname	publisher
축구의 역사	굿스포츠

도서이름에 '축구'가 포함된 출판사를 검색하시오.

```
SELECT      bookname, publisher
FROM        Book
WHERE       bookname LIKE '%축구%';
```

bookname	publisher
축구의 역사	굿스포츠
축구아는 사람	나무수
축구의 이해	대한미디어

SELECT 문

❖ WHERE 조건

도서이름의 왼쪽 두 번째 위치에 '구'라는 문자열을 갖는 도서를 검색하시오.

```
SELECT    *
FROM      Book
WHERE     bookname LIKE '_구%';
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000

표 3-6 와일드 문자의 종류

와일드 문자	의미	사용 예
+	문자열을 연결	'골프 ' + '바이블' : '골프 바이블'
%	0개 이상의 문자열과 일치	'%축구%' : 축구를 포함하는 문자열
[ ]	1개의 문자와 일치	'[0-5]%' : 0-5 사이 숫자로 시작하는 문자열
[^]	1개의 문자와 불일치	'[^0-5]%' : 0-5 사이 숫자로 시작하지 않는 문자열
_	특정 위치의 1개의 문자와 일치	'_구%' : 두 번째 위치에 '구'가 들어가는 문자열

SELECT 문

❖ WHERE 조건

■ 복합조건

축구에 관한 도서 중 가격이 20,000원 이상인 도서를 검색하시오.

```
SELECT    *
FROM      Book
WHERE     bookname LIKE '%축구%' AND price >= 20000;
```

bookid	bookname	publisher	price
3	축구의 이해	대한미디어	22000

출판사가 '굿스포츠' 혹은 '대한미디어'인 도서를 검색하시오.

```
SELECT    *
FROM      Book
WHERE     publisher='굿스포츠'  OR  publisher='대한미디어';
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000

SELECT 문

❖ ORDER BY

도서를 이름순으로 검색하고 bookname을 bname으로 변경한 컬럼을 추가하세요.

```
SELECT    *, bookname as bname
FROM      Book
ORDER BY  bookname;
```

※ **SELECT** 에서 * 는 모든 컬럼을 의미합니다.

bookid	bookname	publisher	price	bname
10	Olympic Champions	Pearson	13000	Olympic Champions
4	골프 바이블	대한미디어	35000	골프 바이블
8	야구를 부탁해	이상미디어	13000	야구를 부탁해
7	야구의 추억	이상미디어	20000	야구의 추억
6	역도 단계별기술	굿스포츠	6000	역도 단계별기술
9	올림픽 이야기	삼성당	7500	올림픽 이야기
2	축구아는 사람	나무수	13000	축구아는 사람
1	축구의 역사	굿스포츠	22000	축구의 역사
3	축구의 이해	대한미디어	22000	축구의 이해
5	피겨 교본	굿스포츠	8000	피겨 교본

bname 컬럼은 첫번째로 배치하세요.

```
SELECT    bookname as bname, b.*
FROM      Book as b
ORDER BY  bookname;
```

SELECT 문

❖ ORDER BY_도서를 이름순으로 보고 싶다

도서를 가격순으로 검색하고, 가격이 같으면 이름순으로 검색하시오.

```
SELECT      *
FROM        Book
ORDER BY    price, bookname;
```

bookid	bookname	publisher	price
6	역도 단계별기술	굿스포츠	6000
1	축구의 역사	굿스포츠	7000
9	올림픽 이야기	삼성당	7500
5	피겨 교본	굿스포츠	8000
10	Olympic Champions	Pearson	13000
8	야구를 부탁해	이상미디어	13000
2	축구아는 여자	나무수	13000
7	야구의 추억	이상미디어	20000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000

SELECT 문

❖ ORDER BY_도서를 이름순으로 보고 싶다

도서를 가격의 내림차순으로 검색하시오. 만약 가격이 같다면 출판사의 오름차순으로 검색한다.

```
SELECT      *
FROM        Book
ORDER BY    price DESC, publisher ASC;
```

bookid	bookname	publisher	price
4	골프 바이블	대한미디어	35000
3	축구의 이해	대한미디어	22000
7	야구의 추억	이상미디어	20000
10	Olympic Champions	Pearson	13000
2	축구아는 여자	나무수	13000
8	야구를 부탁해	이상미디어	13000
5	피겨 교본	굿스포츠	8000
9	올림픽 이야기	삼성당	7500
1	축구의 역사	굿스포츠	7000
6	역도 단계별기술	굿스포츠	6000

SQL 문제

1. 박지성 고객(customer)의 전화번호(phone)를 찾으시오.

```
SELECT    phone
FROM      Customer
WHERE     name = ___;
```

2. 도서(book) 피겨 교본의 출판사(publisher)를 찾으시오.

```
SELECT    publisher
FROM      book
WHERE     bookname = ___;
```

3. 가격(price)이 8000원 이상인 도서(book)를 찾으시오.

```
SELECT    *
FROM      book
WHERE     price >= ___;
```

SQL 문제

4. 가격이 8,000원 이상 15,000원 이하인 도서(book)를 찾으시오.

```
SELECT   *
FROM     Book
WHERE    price BETWEEN ___ AND ___;
          -- price >= 8000 AND price <= 15,000
```

5. 도서(book) 이름에 '역사 ' 가 포함된 도서를 찾으시오.

```
SELECT   *
FROM     book
WHERE    bookname like ____;
```

6. 도서(book) 이름에 왼쪽 3번째 위치에 '의 ' 라는 문자열을 같은 도서를 찾으시오.

```
SELECT   *
FROM     book
WHERE    bookname LIKE ___;
```

집계 함수와 GROUP BY

❖ 집계 함수

고객이 주문한 도서의 총 판매액을 구하시오.

```
SELECT    SUM(saleprice)
FROM      Orders;
```

SUM(saleprice)
118000

※ **의미 있는 열 이름을 출력하고 싶으면 속성이름의 별칭을 지칭하는 AS 키워드를 사용하여 열 이름을 부여한다. 테이블 이름을 줄이기 위해서도 사용할 수 있다.**

```
SELECT    SUM(saleprice) AS 총매출
FROM      Orders;
```

총매출
118000

```
SELECT    SUM(O.saleprice) AS 총매출
FROM      Orders AS O;
```

총매출
118000

집계 함수와 GROUP BY

❖ 집계 함수

2번 김연아 고객이 주문한 도서의 총 판매액을 구하시오.

```
SELECT    SUM(saleprice) AS 총매출
FROM      Orders
WHERE     custid=2;
```

총매출
15000

고객이 주문한 도서의 총 판매액, 평균값, 최저가, 최고가를 구하시오.

```
SELECT    SUM(saleprice) AS Total,
          AVG(saleprice) AS Average,
          MIN(saleprice) AS Minimum,
          MAX(saleprice) AS Maximum
FROM      Orders;
```

	Total	Average	Minimum	Maximum
▶	118000	11800.0000	6000	21000

집계 함수와 GROUP BY

❖ 집계 함수

마당서점의 도서 판매 건수를 구하시오.

```
SELECT    COUNT(*)
FROM      Orders;
```

	COUNT(*)
▶	10

표 3-7 집계 함수의 종류

집계 함수	문법	사용 예
SUM	SUM([ALL | DISTINCT] 속성이름)	SUM(price)
AVG	AVG([ALL | DISTINCT] 속성이름)	AVG(price)
COUNT	COUNT({[[ALL | DISTINCT] 속성이름] | *})	COUNT(*)
MAX	MAX([ALL | DISTINCT] 속성이름)	MAX(price)
MIN	MIN([ALL | DISTINCT] 속성이름)	MIN(price)

집계 함수와 GROUP BY

❖ GROUP BY

고객별로 주문한 도서의 총 수량과 총 판매액을 구하시오.

```
SELECT    custid, COUNT(*) AS 도서수량, SUM(saleprice) AS 총액
FROM      Orders
GROUP BY custid;
```

custid	도서수량	총액
1	3	39000
2	2	15000
3	3	31000
4	2	33000

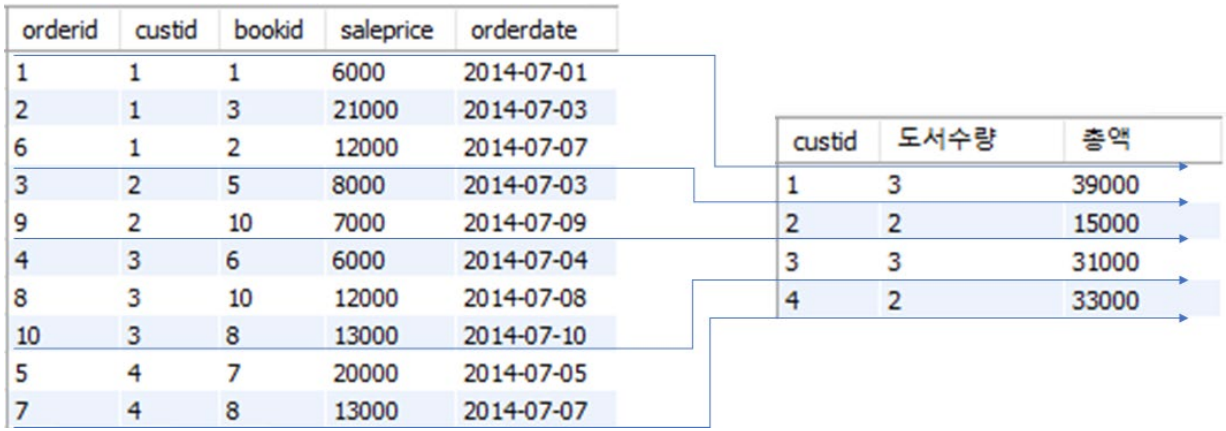


그림 3-18 **GROUP BY 절의 수행**

집계 함수와 GROUP BY

❖ GROUP BY

가격이 8,000원 이상인 도서를 구매한 고객에 대하여 고객별 주문 도서의 총 수량을 구하시오. 단, 두 권 이상 구매한 고객만 구한다.

```
SELECT      custid, COUNT(*) AS 도서수량
FROM        Orders
WHERE       saleprice >= 8000
GROUP BY    custid
HAVING      count(*) >= 2;
```

	custid	도서수량
▶	1	2
	4	2
	3	2

집계 함수와 GROUP BY

❖ GROUP BY 집계 대상에 NULL이 있을 때

질의(etc) col1을 기준으로 GROUP BY 했을 때 col3, col4 값을 모든 집계함수 (COUNT, MIN, Max, SUM, AVG)의 값을 구하시오.

```
SELECT      col1,  count(col3), count(col4),
                   min(col3), min(col4),
                   max(col3), min(col4),
                   sum(col3), sum(col4),
                   avg(col3), avg(col4)

FROM        exaggfn

GROUP BY    col1
```

id	col1	col2	col3	col4
5	col1	NULL	4	NULL
6	col1		NULL	NULL
1	col1	row1	1	NULL
2	col1	row2	2	NULL
3	col1	row3	3	NULL
4	col1	row4	NULL	NULL

(4 + 1 + 2 + 3) / 4
= 2.5

col1	count(col3)	min(col3)	max(col3)	sum(col3)	avg(col3)	count(col4)	min(col4)	max(col4)	sum(col4)	avg(col4)
col1	4	1	4	10	2.5000	0	NULL	NULL	NULL	NULL

집계 함수와 GROUP BY

표 3-8 GROUP BY와 HAVING 절의 문법과 주의사항

문법	주의사항
GROUP BY <속성>	GROUP BY로 투플을 그룹으로 묶은 후 SELECT 절에는 GROUP BY에서 사용한 <속성>과 집계함수만 나올 수 있음 ▪ 맞는 예 SELECT custid, SUM(saleprice) FROM Orders GROUP BY custid; • 틀린 예 SELECT bookid, SUM(saleprice) /* SELECT 절에 bookid 속성이 올 수 없다 */ FROM Orders GROUP BY custid;
HAVING <검색조건>	WHERE 절과 HAVING 절이 같이 포함된 SQL 문은 검색조건이 모호해질 수 있음. HAVING 절은 ① 반드시 GROUP BY 절과 같이 작성해야 하고 ② WHERE 절보다 뒤에 나와야 함. 그리고 ③ <검색조건>에는 SUM, AVG, MAX, MIN, COUNT와 같은 집계함수가 와야 함. • 맞는 예 SELECT custid, COUNT(*) AS 도서수량 FROM Orders WHERE saleprice >= 8000 GROUP BY custid HAVING COUNT(*) >= 2; • 틀린 예 SELECT custid, COUNT(*) AS 도서수량 FROM Orders HAVING COUNT(*) >= 2 /* 순서가 틀렸다 */ WHERE saleprice >= 8000 GROUP BY custid;

마당서점의 데이터

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구아는 사람	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

Book 테이블

orderid	custid	bookid	saleprice	orderdate
1	1	1	6000	2014-07-01
2	1	3	21000	2014-07-03
3	2	5	8000	2014-07-03
4	3	6	6000	2014-07-04
5	4	7	20000	2014-07-05
6	1	2	12000	2014-07-07
7	4	8	13000	2014-07-07
8	3	10	12000	2014-07-08
9	2	10	7000	2014-07-09
10	3	8	13000	2014-07-10

Orders 테이블

custid	name	address	phone
1	박지성	영국 맨체스타	000-5000-0001
2	김연아	대한민국 서울	000-6000-0001
3	장미란	대한민국 강원도	000-7000-0001
4	추신수	미국 클리블랜드	000-8000-0001
5	박세리	대한민국 대전	NULL

Customer 테이블

마당서점의 데이터

Book(bookid, bookname, publisher, price)

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000

Orders(orderid, custid, bookid, saleprice, orderdate)

orderid	custid	bookid	saleprice	orderdate
1	1	1	6000	2014-07-01

Customer(custid, name, address, phone)

custid	name	address	phone
1	박지성	영국 맨체스타	000-5000-0001

마당서점의 데이터 테이블 관계

두 개 이상 테이블에서 SQL 질의

- **Customer 테이블을 Orders 테이블과 조건 없이 연결해보자. Customer와 Orders 테이블의 합체 결과 투플의 개수는 고객이 다섯 명이고 주문이 열 개이므로 5×10 해서 50이 된다.**

```
SELECT    *
FROM      Customer, Orders;
```

custid	name	address	phone	orderid	custid	bookid	saleprice	orderdate
1	박지성	영국 맨체스타	000-5000-0001	1	1	1	6000	2014-07-01
2	김연아	대한민국 서울	000-6000-0001	1	1	1	6000	2014-07-01
3	장미란	대한민국 강원도	000-7000-0001	1	1	1	6000	2014-07-01
4	추신수	미국 클리블랜드	000-8000-0001	1	1	1	6000	2014-07-01
5	박세리	대한민국 대전	NULL	1	1	1	6000	2014-07-01
1	박지성	영국 맨체스타	000-5000-0001	2	1	3	21000	2014-07-03
2	김연아	대한민국 서울	000-6000-0001	2	1	3	21000	2014-07-03
3	장미란	대한민국 강원도	000-7000-0001	2	1	3	21000	2014-07-03
4	추신수	미국 클리블랜드	000-8000-0001	2	1	3	21000	2014-07-03
5	박세리	대한민국 대전	NULL	2	1	3	21000	2014-07-03
1	박지성	영국 맨체스타	000-5000-0001	3	2	5	8000	2014-07-03
2	김연아	대한민국 서울	000-6000-0001	3	2	5	8000	2014-07-03
3	장미란	대한민국 강원도	000-7000-0001	3	2	5	8000	2014-07-03
4	추신수	미국 클리블랜드	000-8000-0001	3	2	5	8000	2014-07-03
5	박세리	대한민국 대전	NULL	3	2	5	8000	2014-07-03
1	박지성	영국 맨체스타	000-5000-0001	4	3	6	6000	2014-07-04
2	김연아	대한민국 서울	000-6000-0001	4	3	6	6000	2014-07-04
3	장미란	대한민국 강원도	000-7000-0001	4	3	6	6000	2014-07-04
4	추신수	미국 클리블랜드	000-8000-0001	4	3	6	6000	2014-07-04
			··· 중략 ···					
1	박지성	영국 맨체스타	000-5000-0001	10	3	8	13000	2014-07-10
2	김연아	대한민국 서울	000-6000-0001	10	3	8	13000	2014-07-10
3	장미란	대한민국 강원도	000-7000-0001	10	3	8	13000	2014-07-10
4	추신수	미국 클리블랜드	000-8000-0001	10	3	8	13000	2014-07-10
5	박세리	대한민국 대전	NULL	10	3	8	13000	2014-07-10

그림 3-16 Customer와 Orders 테이블의 합체

두 개 이상 테이블에서 SQL 질의

❖ 조인_2개의 테이블을 합체해보자

고객과 고객의 주문에 관한 데이터를 모두 보이시오.

```
SELECT    *
FROM      Customer, Orders
WHERE     Customer.custid = Orders.custid;
```

custid	name	address	phone	orderid	custid	bookid	saleprice	orderdate
1	박지성	영국 맨체스타	000-5000-0001	1	1	1	6000	2014-07-01
1	박지성	영국 맨체스타	000-5000-0001	2	1	3	21000	2014-07-03
2	김연아	대한민국 서울	000-6000-0001	3	2	5	8000	2014-07-03
3	장미란	대한민국 강원도	000-7000-0001	4	3	6	6000	2014-07-04
4	추신수	미국 클리블랜드	000-8000-0001	5	4	7	20000	2014-07-05
1	박지성	영국 맨체스타	000-5000-0001	6	1	2	12000	2014-07-07
4	추신수	미국 클리블랜드	000-8000-0001	7	4	8	13000	2014-07-07
3	장미란	대한민국 강원도	000-7000-0001	8	3	10	12000	2014-07-08
2	김연아	대한민국 서울	000-6000-0001	9	2	10	7000	2014-07-09
3	장미란	대한민국 강원도	000-7000-0001	10	3	8	13000	2014-07-10

두 개 이상 테이블에서 SQL 질의

❖ 조인_2개의 테이블을 합체해보자

고객과 고객의 주문에 관한 데이터를 고객번호 순으로 정렬하여 보이시오.

```
SELECT      *
FROM        Customer, Orders
WHERE       Customer.custid = Orders.custid
ORDER BY    Customer.custid;
```

custid	name	address	phone	orderid	custid	bookid	saleprice	orderdate
1	박지성	영국 맨체스타	000-5000-0001	6	1	2	12000	2014-07-07
1	박지성	영국 맨체스타	000-5000-0001	1	1	1	6000	2014-07-01
1	박지성	영국 맨체스타	000-5000-0001	2	1	3	21000	2014-07-03
2	김연아	대한민국 서울	000-6000-0001	3	2	5	8000	2014-07-03
2	김연아	대한민국 서울	000-6000-0001	9	2	10	7000	2014-07-09
3	장미란	대한민국 강원도	000-7000-0001	4	3	6	6000	2014-07-04
3	장미란	대한민국 강원도	000-7000-0001	8	3	10	12000	2014-07-08
3	장미란	대한민국 강원도	000-7000-0001	10	3	8	13000	2014-07-10
4	추신수	미국 클리블랜드	000-8000-0001	5	4	7	20000	2014-07-05
4	추신수	미국 클리블랜드	000-8000-0001	7	4	8	13000	2014-07-07

두 개 이상 테이블에서 SQL 질의

❖ 조인_2개의 테이블을 합체해보자

고객의 이름과 고객이 주문한 도서의 판매가격을 검색하시오.

```
SELECT    name, saleprice
FROM      Customer, Orders
WHERE     Customer.custid =Orders.custid;
```

name	saleprice
박지성	6000
박지성	21000
김연아	8000
장미란	6000
추신수	20000
박지성	12000
추신수	13000
장미란	12000
김연아	7000
장미란	13000

두 개 이상 테이블에서 SQL 질의

❖ 조인_2개의 테이블을 합체해보자

고객별로 주문한 모든 도서의 총 판매액을 구하고, 고객별로 정렬하시오.

```
SELECT      name, SUM(saleprice)
FROM        Customer, Orders
WHERE       Customer.custid =Orders.custid
GROUP BY Customer.name
ORDER BY Customer.name;
```

name	SUM(saleprice)
김연아	15000
박지성	39000
장미란	31000
추신수	33000

두 개 이상 테이블에서 SQL 질의

❖ 조인_2개의 테이블을 합체해보자

고객의 이름과 고객이 주문한 도서의 이름을 구하시오.

CUSTID	NAME	ADDRESS	PHONE
1	박지성	영국 맨체스타	000-5000-0001
2	김연아	대한민국 서울	000-6000-0001
3	장미란	대한민국 강원도	000-7000-0001
4	추신수	미국 클리블랜드	000-8000-0001
5	박세리	대한민국 대전	(null)

BOOKID	BOOKNAME	PUBLISHER	PRICE
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	역도 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

ORDERID	CUSTID	BOOKID	SALEPRICE	ORDERDATE
1	1	1	6000	14/07/01
2	1	3	21000	14/07/03
3	2	5	8000	14/07/03
4	3	6	6000	14/07/04
5	4	7	20000	14/07/05
6	1	2	12000	14/07/07
7	4	8	13000	14/07/07
8	3	10	12000	14/07/08
9	2	10	7000	14/07/09
10	3	8	13000	14/07/10

그림 3-20 마당서점 데이터 간의 연결

두 개 이상 테이블에서 SQL 질의

❖ 조인_2개의 테이블을 합체해보자

고객의 이름과 고객이 주문한 도서의 이름을 구하시오.

```
SELECT    Customer.name, Book.bookname
FROM      Customer, Orders, Book
WHERE     Customer.custid =Orders.custid
          AND Orders.bookid =Book.bookid;
```

name	bookname
박지성	축구의 역사
박지성	축구의 이해
김연아	피겨 교본
장미란	역도 단계별기술
추신수	야구의 추억
박지성	축구아는 여자
추신수	야구를 부탁해
장미란	Olympic Champions
김연아	Olympic Champions
장미란	역도를 부탁해

두 개 이상 테이블에서 SQL 질의

❖ 조인_2개의 테이블을 합체해보자

가격이 20,000원인 도서를 주문한 고객의 이름과 도서의 이름을 구하시오.

```
SELECT    Customer.name, Book.bookname
FROM      Customer, Orders, Book
WHERE     Customer.custid =Orders.custid AND Orders.bookid =Book.bookid
          AND Book.price =20000;
```

name	bookname
추신수	야구의 추억

두 개 이상 테이블에서 SQL 질의

❖ 조인_2개의 테이블을 합체해보자

■ 외부조인

도서를 구매하지 않은 고객을 포함하여 고객의 이름과 고객이 주문한 도서의 판매가격을 구하시오.

```
SELECT    Customer.name, saleprice
FROM      Customer LEFT OUTER JOIN Orders
                ON Customer.custid =Orders.custid;
```

name	saleprice
박지성	6000
박지성	21000
김연아	8000
장미란	6000
추신수	20000
박지성	12000
추신수	13000
장미란	12000
김연아	7000
장미란	13000
박세리	NULL

두 개 이상 테이블에서 SQL 질의

❖ 조인_2개의 테이블을 합체해보자

표 3-9 조인 문법

명령	문법	설명
일반 조인	SELECT <속성들> FROM 테이블1, 테이블2 WHERE <조인조건> AND <검색조건>	SQL 문에서는 주로 동등조인을 사용함. 두 가지 문법 중 하나를 사용할 수 있음.
	SELECT <속성들> FROM 테이블1 INNER JOIN 테이블2 ON <조인조건> WHERE <검색조건>	
외부조인	SELECT <속성들> FROM 테이블1 {LEFT |RIGHT |FULL [OUTER]} JOIN 테이블2 ON <조인조건> WHERE <검색조건>	외부조인은 FROM 절에 조인 종류를 적고 ON을 이용하여 조인조건을 명시함.